

Programmazione Concorrente e Distribuita

Ingegneria e Scienze Informatiche - UNIBO
a.a 2025/2026
Prof. Alessandro Ricci

[module 3.2]

ACTORS

ACTORS

Voleva catturare il modo di descrivere
computazioni che mettersero al centro lo scambio
di messaggi necessariamente asincrono

- Originally introduced by Carl Hewitt and colleagues at MIT in 70ies
 - AI context
- Developed by Gul Agha, Akinori Yonezawa et al. in 80ies and 90ies
 - idea di attore
 - **unification of OOP and concurrency**
 - Concurrent Object Oriented Programming
 - [YON-87][AGHA-90]
 - many languages & frameworks developed
 - ACT++, SALSA, Kilim, ABCL family, E, AmbientTalk, ActorFoundry,...
 - recent paper: [ACTORS][Hewitt's pure perspective]
- Playing a major role in the mainstream nowadays
 - as **an alternative model to multi-threaded programming**
 - Erlang, Scala/Akka actors, HTML5 Web Workers, DART isolates, etc.

Modello ad attori pensato per la
programmazione concorrente, non
più per AI come idea iniziale di Hewitt,
per pensare ad un attore come un
unione di concorrenza e OO

ACTOR MODEL

- The Actor Model is a mathematical theory that treats “Actors” as the universal primitives of concurrent digital computation
 - see “*Actor Model of Computation: Scalable Robust Information Systems*” by Carl Hewitt

ACTOR LIBRARIES AND FRAMEWORKS IN MAINSTREAM LANGUAGES

- Many actor frameworks based on familiar languages (from [KAR-2009]):
 - C/C++ (Act++ [8], Broadway [9], Thal [10]),
 - Smalltalk (Actalk [11]),
 - Python (Stackless Python [12], Parley [13]),
 - Ruby (Stage [14]),
 - .NET (Microsoft's Asynchronous Agents Library [15], Retlang [16])
 - Java/Scala Actors library [17], Kilim [18], Jetlang [19], ActorFoundry [20], Actor Architecture [21], Actors Guild [22], JavAct [23], AJ [24], and Jsasb [25])
- Recent updates
 - http://en.wikipedia.org/wiki/Actor_model

ACTORS: CORE IDEA

Gli attori sono entità primitive che sanno fare 3 cose (linguaggio base ad attori): creare un altro attore, mandare un messaggio e cambiare il proprio stato per riuscire a ricevere messaggi di tipo diverso (a differenza degli oggetti, gli attori possono cambiare interfaccia, quindi cambiare comportamento, e stato interno)

- **Asynchronous message passing** among autonomous purely reactive objects called **actors**
 - **everything is an actor** Modelliamo la computazione completamente in termini di attori e scambi di messaggi: al centro ci deve essere lo scambio di messaggi
 - with a unique identifier
 - a unique mailbox where messages are enqueued
 - every interaction takes place as async message passing
 - strongly related to Alan Kay's original OOP idea
 - the key point was message passing
 - <http://lists.squeakfoundation.org/pipermail/squeak-dev/1998-October/017019.html>
 - <http://worrydream.com/EarlyHistoryOfSmalltalk/>
- *Everything - including traditional control structures, can be modeled as patterns of messages among actors*
 - [HEW-77]

ACTORS AS AUTONOMOUS REACTIVE OBJECTS

La differenza tra un oggetto tradizionale e un attore. Mentre nel mondo OOP classico (Java, C++) un oggetto è "passivo", nel modello ad attori l'attore diventa un'entità autonoma e reattiva.

- Actor abstraction
 - ~~computational entities~~ ^{entità computazionali} **encapsulating** a state, behaviour and a ***logical control flow*** (or *thread of control*)
 - classic objects in modern OOP do not encapsulate control flow
- Uncoupling physical vs. **logical concurrency**
 - actors do not necessarily own a physical thread of control (OS threads)
 - you can have a system of N actors (e.g. 100000) running on a machine using M cores/threads (e.g. 8) Gli attori sono entità dinamiche

ACTOR BASIC PRIMITIVES

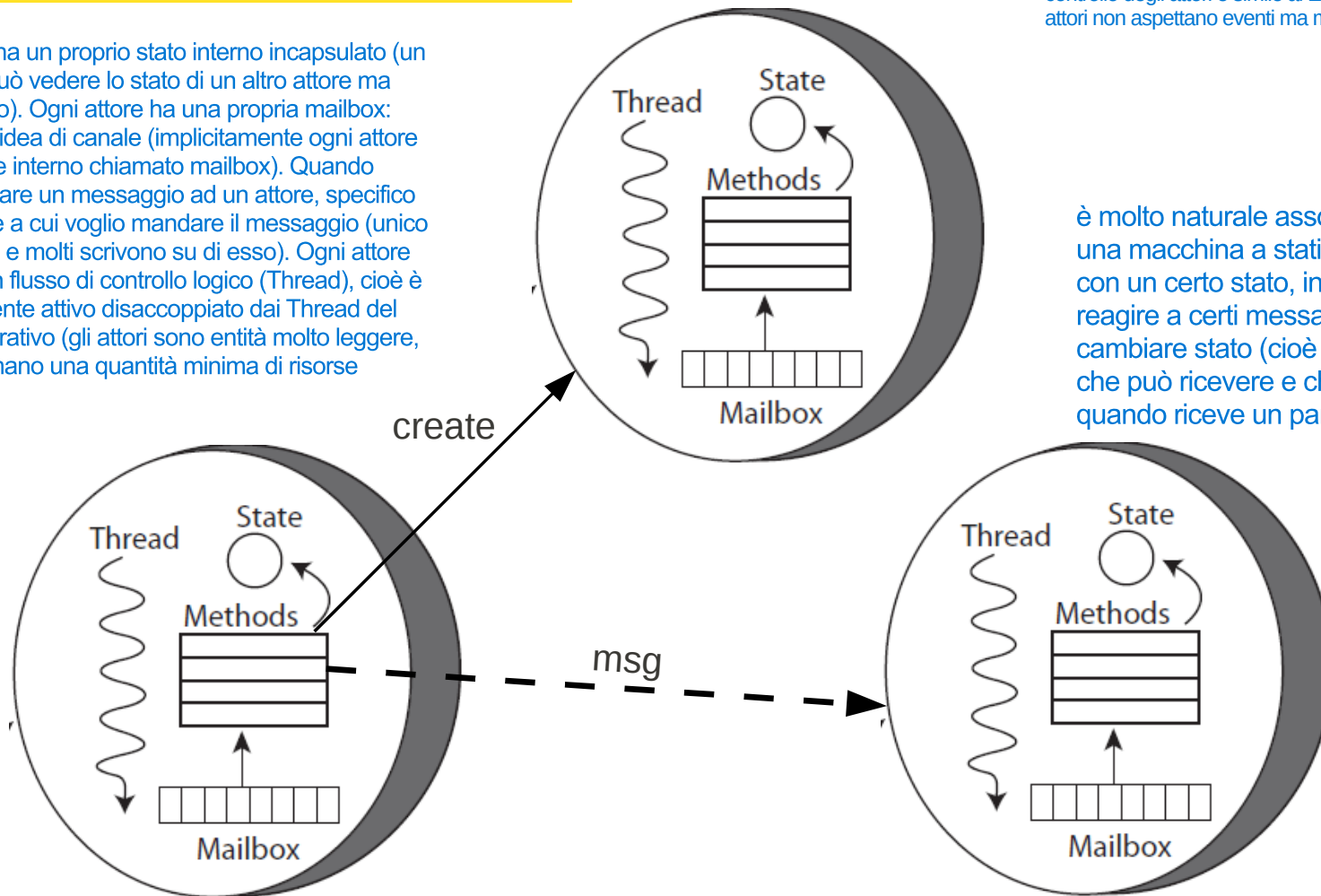
- Only three primitives (actions) to compose an actor behaviour
 - **send** Quando un attore invia un messaggio, non aspetta una risposta. Lo deposita nella mailbox del destinatario e continua immediatamente il suo lavoro.
 - asynchronously sending a message to a specified actor
 - it is to concurrent programming what procedure invocation is to sequential programming
 - **create**
 - create an actor with the specified behavior
 - it is to concurrent programming what procedure abstraction is to sequential programming
 - **become** Grazie a become, un attore può comportarsi come una macchina a stati. Ad esempio, un attore che gestisce un database può essere nello stato Connesso e rispondere alle query, oppure ricevere un messaggio
 - specify a new behavior (local state) to be used by actor to respond to the next message it processed Macchina a Stati Finiti (FSM): La primitiva become trasforma l'attore in una macchina a stati.
 - gives actors a **history-sensitive** behaviour necessary for shared, mutable data objects Poiché l'attore può cambiare comportamento in base a ciò che è successo prima, può modellare oggetti mutabili in modo sicuro senza usare variabili globali o lock.
- An actor can only communicate with actors to which it is connected
 - connected means the actor knows their identifiers
 - id can be exchanged in messages..

ACTOR MODEL

Esempio: Attore Counter che è in grado di ricevere messaggio Inc oppure GetValue e l'attore che riceve il messaggio manderà in esecuzione, se ce l'ha, il metodo legato al messaggio ricevuto (l'architettura di controllo degli attori è simile al Event-loop ed Eventi: gli attori non aspettano eventi ma messaggi)

Ogni attore ha un proprio stato interno incapsulato (un attore non può vedere lo stato di un altro attore ma solo il proprio). Ogni attore ha una propria mailbox: non c'è più l'idea di canale (implicitamente ogni attore ha un canale interno chiamato mailbox). Quando voglio mandare un messaggio ad un attore, specifico l'id dell'attore a cui voglio mandare il messaggio (unico attore riceve e molti scrivono su di esso). Ogni attore incapsula un flusso di controllo logico (Thread), cioè è un componente attivo disaccoppiato dai Thread del sistema operativo (gli attori sono entità molto leggere, cioè consumano una quantità minima di risorse hardware)

è molto naturale associare un attore ad una macchina a stati: un attore parte con un certo stato, in cui è in grado di reagire a certi messaggi, poi può cambiare stato (cioè cambia i messaggi che può ricevere e che cosa fare quando riceve un particolare messaggio)



KEY ASPECTS OF THE SEMANTICS

Queste sono le regole che governano il funzionamento degli attori

Key aspects of the semantics defining the actor behavior

pure reactive behaviour

Questo permette l'efficienza estrema di cui parlavamo prima: milioni di attori possono coesistere perché la maggior parte del tempo sono fermi.

- an actor works only when receiving a message
- no messages => blocked

Sebbene l'attore sia un'entità reattiva, quasi tutti i linguaggi/framework (come Akka o Erlang) prevedono un momento speciale chiamato Inizializzazione, dove si alloca la memoria per l'attore. Viene eseguito un blocco di codice speciale. Qui l'attore può impostare il suo stato iniziale o inviare un messaggio a se stesso per "rompere il ghiaccio". Una volta terminata l'inizializzazione, l'attore entra nel loop reattivo puro. Da quel momento in poi, se non riceve messaggi, non consuma CPU.

state encapsulation

and behaviour

Se l'Attore A vuole conoscere lo stato dell'Attore B, deve inviargli un messaggio e attendere che B risponda con l'informazione.

- an actor cannot directly access to the inner state of other actors

macro-step semantics (run-to-completion)

Perché è importante? Ti permette di scrivere il codice interno all'attore come se fosse un normale programma single-threaded, senza preoccuparti che qualcuno modifichi i tuoi dati mentre ci stai lavorando.

- once received a message, the corresponding computation is executed completely before serving another message

il messaggio viene accodato (metodi eseguiti atomicamente e devono terminare (no loop infinito))

fairness in message delivering & processing

Se invii un messaggio, il sistema si impegna a recapitarlo (a patto che il destinatario non sia morto o il sistema non sia andato in crash totale) (No Starvation).

- a message sent to an actor is eventually delivered to its destination and processed by the actor

prima o poi

nessuna assunzione su ordine di arrivo messaggi e su quando arriverà

location transparency

il sistema di nomi degli attori mi permette di fare questo

- to send a msg to an actor we don't need to know its location, just its identity

Puoi spostare gli attori da un server all'altro per bilanciare il carico senza dover cambiare una singola riga di codice nel mittente.

ciò mi permette di migrare, a runtime del sistema, l'attore in un'altra zona geografica o server disgiunto, continuando a comunicare con i precedenti attori, senza modificare il codice

REMARKS ABOUT MESSAGES

Non c'è la receive del Message passing model: è implicita

- **Send semantics** La semantica della send garantisce che un messaggio arrivi a destinazione, ma non promette quando.
 - in the basic model messages sent by a send are ^{prima o poi} ~~eventually~~ delivered to the target actor, but no assumption can be done on *how long* it will take
 - **No ordering & msg exchange patterns**
 - no assumption can be done on the order with which messages are received by the target actor Il modello non garantisce in alcun modo l'ordine di arrivo dei messaggi nella mailbox del destinatario.
 - ...even when the two sends are done by the same sender actor, in sequence
 - every kind of ordering (as well as synch and coordinated behavior) must be achieved by means of *patterns of message exchanges* L'attore deve essere progettato per essere "robusto" rispetto all'ordine. Se l'ordine è vitale, deve essere lo sviluppatore a gestirlo (usando dei numeri di sequenza o dei pattern specifici).
- Quindi, qualsiasi forma di sincronizzazione, coordinamento o ordinamento deve essere costruita dallo sviluppatore usando appositi pattern di scambio messaggi (Message Exchange Patterns).

ACTOR MODEL IMPLEMENTATIONS

- Implemented in many recent languages and frameworks
 - Erlang, Scala Actors, Vert.x Verticles, HTML5 Web Worker, Google DART (isolates)...

Vert.x usa un modello di concorrenza basata su Event Loop che è estremamente simile (e spesso sovrapponibile) al Modello ad Attori. Un verticle è quasi identico ad un attore; Event bus è una mailbox distribuita. Però Vert.x: È orientato agli eventi e agli indirizzi, non all'identità come nei modelli ad attori

- However with slightly different features and semantics
 - see [KAR-09]

- Some main important differences
 - explicit* vs. *implicit* message loop
 - pattern-driven receive

Pattern-Driven Receive: Selezione del messaggio in base alla sua struttura (tupla, tipo, guardie) anziché con catene di if-else.
Selective Receive: L'attore prende dalla mailbox solo i messaggi che gli servono subito e lascia gli altri in coda per il futuro.

In alcuni linguaggi, è possibile definire delle callback all'arrivo del messaggio nel top della queue. In altri messaggi, è possibile implementare async-await con l'arrivo dei messaggi

- Explicit Loop (es. Erlang): L'attore decide quando chiamare la receive per estrarre il messaggio dalla mailbox.
- Implicit Loop (es. Akka): Il framework decide per l'attore, consegnando il messaggio e chiamando un handler (es. onReceive).

ENCAPSULATED EVENT LOOP & IMPLICIT RECEIVE

- Actor behaviour ^{è guidato da} ~~governed by~~ an **message-loop (event-loop)**

```
loop {  
  msg <- waitForMsg()  
  handler <- selectHandler(msg)  
  execute (handler)  
}
```

handler deve terminare (no loop infinito) e non deve essere molto pesante così ho un attore reattivo

Il codice rappresenta il ciclo di vita infinito di un attore. Anche se spesso non lo scrivi esplicitamente, ogni attore funziona così:

waitForMsg(): L'attore è in pausa (non consuma CPU). Appena arriva un messaggio nella mailbox, il sistema lo "sveglia".

selectHandler(msg): Il sistema guarda il tipo di messaggio (es. "SpostaSoldi", "StampaReport") e cerca il pezzo di codice (handler) programmato per gestirlo.

execute(handler): Viene eseguita la logica. Qui l'attore può modificare il suo stato, creare altri attori o inviare messaggi.

- Actor behavior defined by the handlers (body)
 - each handler is a sequence of actions modifying the inner state of the actor and possibly sending messages and creating other actors

Actor explicit control architecture ^(il codice del loop sopra)

- embedding the loop out of programmers' code

In molti framework moderni (come Akka), tu come programmatore non scrivi il ciclo loop. Il ciclo è "nascosto" nell'infrastruttura del framework. Tu devi solo definire gli handler (il corpo del comportamento). Non devi preoccuparti di gestire l'attesa del messaggio o la gestione degli errori del loop. Ti concentri solo sulla logica di business. Poiché il loop è gestito dal framework, è garantita la semantica macro-step: il framework si assicura che un handler finisca completamente prima di passare al messaggio successivo.



In pratica, l'attore non "chiama" il sistema per avere lavoro; è il sistema che "spinge" il lavoro (il messaggio) dentro l'attore.

ENCAPSULATED EVENT LOOP & IMPLICIT RECEIVE

- *Macro-step semantics*
 - an handler is executed completely before receiving the next message
 - handlers should have a non-blocking behavior
 - the only “blocking” point is managed by the event loop
- Same semantics of *event-driven architectures*
 - event-driven OS
 - e.g. GUI management
 - modern Web programming
 - e.g. JavaScript with callbacks
 - both client & server (e.g. node.js)

CONCRETE EXAMPLES: ACTOR foundry & AKKA

- ActorFoundry
 - Java-based academic-level framework developed by Agha's group
 - <http://osl.cs.uiuc.edu/software/actor-foundry/>
 - it include some optimizations and idea developed by other research works, such as Kilim
 - see [KAR-09]
- Akka technology
 - rich-full industrial-oriented actor framework for Java and Scala environments
 - <http://akka.io/>
 - including many other features (e.g. Erlang-like fault tolerance,...)
 - superseding Scala Actor lib
 - see Philipp Haller invited at AGERE! workshop 2012 [AGERE]
 - <http://docs.scala-lang.org/overviews/core/actors-migration-guide.html>

TASTE OF ACTORS IN ACTOR-FOUNDRY

```
public class PingActor extends Actor {
    ActorName otherPinger;
    @message
    public void start(ActorName other) {
        otherPinger = other;
        send(otherPinger, "ping", self(), Id.stamp()+"called from " + self());
    }
    @message
    public void ping(ActorName caller, String msg) {
        send(stdout, "println", Id.stamp()+"Received ping (" + msg + ") from " + caller + "...");
        send(caller, "alive", Id.stamp()+self().toString() + " is alive");
    }
    @message
    public void alive(String reply) {
        send(stdout, "println", Id.stamp()+"Received " + reply + " from pinged actor");
    }
}
```

@message rappresentano i message handler
(metodi che gestiscono quel tipo di messaggio)

↳ primitiva di send: l'attore a cui inviare, il messaggio, l'attore che lo manda ed altri parametri

```
public class PingBoot extends Actor {

    @message
    public void boot() throws RemoteCodeException {
        ActorName pinger1 = null;
        ActorName pinger2 = null;

        pinger1 = create(osl.examples.ping.PingActor.class);
        pinger2 = create(osl.examples.ping.PingActor.class);

        send(pinger1, "start", pinger2);
    }
}
```

↳ creo l'attore specificando la sua classe

TASTE OF AKKA ACTORS IN JAVA

- UNTYPED ACTORS

Prima versione di akka (non sfruttavano i tipi)

```
import akka.actor.UntypedActor;
import akka.event.Logging;
import akka.event.LoggingAdapter;
```

```
public class MyUntypedActor extends UntypedActor {
    LoggingAdapter log = Logging.getLogger(getContext().system(), this);

    public void onReceive(Object message) throws Exception {
        if (message instanceof String){
            log.info("Received String message: {}", message);
            getSender().tell("received");
        } else
            unhandled(message);
    }
}
```

estendendo questa classe si doveva fare override di onReceive in cui si specificava il comportamento da avere quando si riceveva un messaggio. La tell era il metodo per inviare un messaggio

TASTE OF AKKA ACTORS IN SCALA - TYPED ACTORS

```
import akka.actor.Actor
import akka.actor.Props
import akka.event.Logging

class MyActor extends Actor {
  val log = Logging(context.system, this)

  def receive = {
    case "test" => log.info("received test")
    case _      => log.info("received unknown message")
  }
}
```

ENCAPSULATED EVENT LOOP & IMPLICIT RECEIVE: REMARKS

- Macro-step semantics - consequences
 - this avoids race conditions and simplifies reasoning about programs
 - ...but it strongly complicates programming
 - “*asynchronous spaghetti*” problem
 - see [RS-12]

“ASYNCHRONOUS SPAGHETTI”

- Problem affecting more generally modern event-driven architectures based on *callbacks* and *event loops*
 - see module about asynchronous programming
- Application logic broken in a unstructured set of handlers (or callbacks)
 - Il termine "Asynchronous Spaghetti" descrive una situazione in cui la logica del tuo programma, invece di seguire un flusso lineare e leggibile, viene frammentata in mille piccoli pezzi (gli handler) difficili da seguire mentalmente.
 - each reacting to some event
 - e.g. receipt of a message
- example
 - actor program to evaluate the expression $\sin(x) \cdot \cos(y)$ exploiting multiple actors
 - L'attore Coordinatore deve "ricordarsi" lo stato. Se riceve prima il risultato del seno, deve salvarlo in una variabile e aspettare il coseno. Deve gestire i messaggi come se fossero pezzi di un puzzle che arrivano in ordine sparso. Leggendo il codice di un singolo handler, perdi la visione d'insieme di cosa sta cercando di fare l'intero sistema.
 - hp: x and y are known to one initial actor and we have *sin* actor and *cos* actor
- Proposed solutions
 - *promises*, promise *pipelines*

SYNCHRONIZATION & MSG ORDERING

- Synchronization in actors is achieved through communication
- Two commonly used communication patterns: (Patterns of Message exchange)
 - 1 – **Remote Procedure Call (RPC) -like messaging**
 - sending a request and waiting the reply before proceeding
 - 2 – **local synchronization constraints** → Vengono definite delle guardie (predicati logici) basate sullo stato attuale dell'attore e sul tipo di messaggio in arrivo. Il messaggio viene elaborato solo se la guardia restituisce true.
 - selezionare quando ~~selecting when~~ (=guards) ricevere i messaggi ~~receiving messages~~
- Language constructs are typically provided to actor programmers to specify such patterns
 - such lang constructs are definable anyway in terms of primitive actor constructs
 - ... but providing them as first-class linguistic objects simplifies programming Fornire questi pattern come costrutti linguistici nativi (annotazioni, primitive di call o stash) semplifica drasticamente la programmazione e migliora la leggibilità del codice.

1 RPC-like MESSAGING

come simulare il comportamento di una chiamata a funzione tradizionale (dove chiami e aspetti il risultato) all'interno di un sistema che, per natura, non aspetta mai.

- Can be implemented as a protocol.
 - e.g [ACTORS]:
 - the client actor sends a request;
 - the client then checks incoming messages;
 - if the incoming message corresponds to the reply to its request, the client takes the appropriate action
 - if an incoming message does not correspond to the reply to its request, the message must be handled
 - for example, by being buffered for later processing
 - the client continues to check messages for the reply
- Direct support provided by frameworks
 - `call` primitive in ActorFoundry

2 MESSAGE ORDERING PROBLEM

- il numero di possibili ordini di arrivo è esponenziale rispetto ai messaggi "pendenti" (quelli inviati ma non ancora ricevuti).
L'asincronia **Asynchrony => the number of possible orderings in which messages may arrive is exponential in the number of messages that are 'pending' at any time**
il numero di possibili ordinamenti con cui i messaggi possono arrivare è esponenziale rispetto al numero di messaggi che sono stati inviati ma non ancora ricevuti.
 - i.e., **messages that have been sent but have not been received** Perché succede? I messaggi possono percorrere rotte di rete diverse, essere ritardati da un intoppo su un router o essere gestiti da thread diversi sul server di destinazione.
- potrebbe non essere a conoscenza **A sender may be unaware of the state of the actor receiver of the message => the recipient may not be in a state where it can process the message it is receiving.** destinatario
L'Attore A non sa se l'Attore B è pronto, se è sovraccarico o se si trova in uno stato logico incompatibile con quel messaggio.
 - e.g. a spooler may not have a job when some printer requests one.
- La necessità di garantire specifici ordinamenti porta a una notevole **The need for such orderings leads to considerable complexity in concurrent programs**
introducendo spesso **often introducing bugs or inefficiencies due to suboptimal implementation strategies**
- Complessità: Il programmatore deve scrivere codice per gestire messaggi che arrivano "troppo presto" o "troppo tardi".
Come si risolve questo problema nel Modello ad Attori?
 - Local Synchronization Constraints
 - Stashing (es. Akka)
 - Cambio dinamico di comportamento (become / unbecome o Pattern Matching): l'attore ridefinisce la propria funzione di risposta ai messaggi in base allo stato in cui si trova.

Se la gestione di questi messaggi "arrivati troppo presto" o "fuori ordine" viene implementata manualmente all'interno dell'attore (ad esempio salvandoli in una lista/buffer interno all'interno dei singoli metodi dell'attore), si creano due gravi problemi: Violazione della separazione delle responsabilità (Separation of Concerns) e Rischio di Bug e Inefficienze.

2 LOCAL SYNCHRONIZATION CONSTRAINTS

- Se i messaggi in sospeso vengono memorizzati esplicitamente in un buffer all'interno del corpo di un attore, il codice che specifica la funzionalità dell'attore si mescola con la logica che determina l'ordine con cui l'attore elabora i messaggi.
- If pending messages are buffered explicitly inside the body of an actor, the code specifying the functionality (the *how* or *representation*) of the actor is mixed with the logic determining the order in which the actor processes the messages (the *when*)
 - In un attore scritto "male" (o in modo ingenuo), troverai due tipi di codice mescolati:
 - The How (Funzionalità): Il codice che effettivamente esegue il lavoro (es. "sottrai 100 euro dal conto").
 - The When (Sincronizzazione): Il codice che controlla se è il momento giusto per farlo (es. "se il messaggio è 'Preleva' ma il saldo è zero, metti il messaggio in un buffer e aspetta").
 - such mixing violates the software *principle of separation of concerns*.
 - Researchers have proposed various constructs to enable programmers to specify the correct orderings in a modular and abstract way
 - specifically, as logical formulae (predicates) over the state of an actor and the type of messages.
 - Many actor languages and frameworks provide such constructs
 - local synchronization constraints in ActorFoundry

2 LSC IN ACTOR FOUNDRY

Consentono di specificare delle condizioni/guardie sullo stato dell'attore. Se la condizione per un certo messaggio non è soddisfatta, il runtime sospende il messaggio ponendolo automaticamente in una coda temporanea (save queue) e lo ripropone all'attore non appena lo stato cambia.

- **Enabling/disabling condition** specified for handlers
 - **by means of annotations**

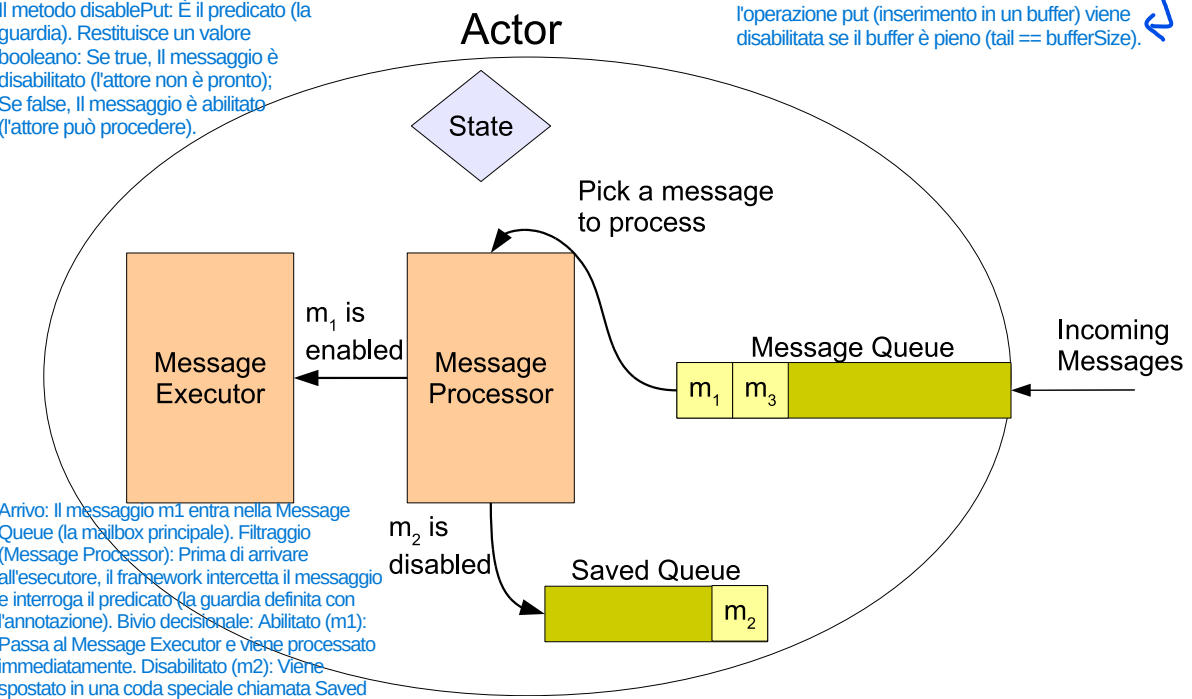
Condizioni di attivazione/disattivazione

```
@Disable(messageName = "put")
public Boolean disablePut(Integer x) {
    if (bufferReady) {
        return (tail == bufferSize);
    } else return true;
}
```

Indica al framework che il messaggio "put" non deve essere sempre accettato.

Il metodo disablePut: È il predicato (la guardia). Restituisce un valore booleano: Se true, il messaggio è disabilitato (l'attore non è pronto); Se false, il messaggio è abilitato (l'attore può procedere).

l'operazione put (inserimento in un buffer) viene disabilitata se il buffer è pieno (tail == bufferSize).



In this code, the constraint returns true if the put message cannot be processed in the actor's current state. At runtime, a message is processed if it is not disabled i.e., no constraint returns true for the message. A disabled message is placed in a queue called save queue for later processing.

Per risolvere questo problema, sono stati introdotti i vincoli di sincronizzazione locali. L'idea è di spostare la logica del "quando" fuori dal corpo dell'attore, rendendola dichiarativa.

Invece di scrivere:
if (saldo > 0) { elabora(messaggio); } else { metti_in_coda(messaggio); }
Si definiscono dei Predicati (formule logiche) che fungono da guardie:
Messaggio: Preleva(somma)
Condizione (Guardia): stato.saldo >= somma

Il framework dell'attore controlla automaticamente la guardia prima di consegnare il messaggio all'attore. Se la condizione è falsa, il messaggio rimane nella mailbox o in un buffer speciale, e l'attore non lo vede nemmeno finché la condizione non diventa vera.

2 AKKA SOLUTION: STASHING

Mi permette di poter mettere da parte dei messaggi che mi arrivano in un certo momento che io non voglio elaborarli, ma mi interessano per il futuro quando sarò in un altro stato per poi poterli riprendere quanto transito in quello stato (cioè quando faccio la become in un nuovo behavior dove quei messaggi che avevo ricevuto mi interessano ora): perchè, se non ho questo meccanismo, quei messaggi che non hanno l'handler, perchè non sono in quello stato in cui li posso eseguire, verrebbero eliminati

- Using a separate implicit queue
 - *stash* primitive to store received messages for later processing
 - *unstash* to bring back stashed msgs in the main queue

stash(): Prende il messaggio che l'attore sta analizzando in quel preciso momento e lo sposta dalla mailbox principale alla stash queue (la coda di riserva).

```
import akka.actor.Stash
class ActorWithProtocol extends Actor with Stash {
  def receive = {
    case "open" =>
      unstashAll()
      context.become({
        case "write" => // do writing...
        case "close" =>
          unstashAll()
          context.unbecome()
        case msg => stash()
      }, discardOld = false) // stack on top instead of replacing
    case msg => stash()
  }
}
```

Questa slide presenta la soluzione pratica di Akka (il framework più usato in Java/Scala) al problema dei "Vincoli di Sincronizzazione" che abbiamo visto nelle slide precedenti.

In breve, lo Stashing è la soluzione di Akka al problema del:
"Cosa faccio se ricevo un messaggio che non posso (o non voglio) gestire in questo preciso momento?"

unstashAll() (o unstash())
Cosa fa: Svuota la stash queue e rimette tutti i messaggi salvati in testa alla mailbox principale, rispettando l'ordine originale in cui erano stati messi da parte.

ACTOR IDIOMS

Design Pattern

che coinvolgono più attori che interagiscono tra loro

- ~~Recurrent design schema involving multiple interacting actors~~
 - Future
 - Serializer
 - Fork-Join
 - ...
- An overview
 - “Actor Idioms” by Dale Schumacher - AGERE! 2012 - <http://dl.acm.org/citation.cfm?id=2414655>

FUTURES

- Generally speaking, future (and promises) refer to objects that acts as a proxy for a result that is initially unknown because the computation of its value is yet incomplete.
 - setting the value of a future is also called resolving, fulfilling, or binding it
 - *promise* is the mechanism which sets the Future
- In the actor model, a future is represented by an actor, featuring three states:
 - Initially, the future has no value and no customers waiting for the value
 - If Customers arrive to read the value, they are queued until the value is available I messaggi vengono presi subito dalla mailbox e gli ID (o ActorRef) degli attori interessati vengono salvati in una struttura dati interna all'attore "Future" (es. una lista, un set o un array).
 - When the value is available, any waiting Customers are notified, and any subsequent
- The identifier of the future actor is released immediately to a Customer
- Once set, the value of a future never changes

FUTURE: KEY POINTS

- Enhancing parallelism
- Mechanism used to implement synchronous forms of communication using asynch msg passing maximizing concurrency
 - the future can be passed to other actors - which may send message to it - *before* it is resolved
- Actor languages providing a direct support to futures/promises
 - E language, AmbientTalk

MESSAGING PATTERNS

- Messaging Patterns [VER16]
 - **Messaging Channels**
 - Point-to-Point, Publish-Subscribe, Invalid Message, Dead Letter, Channel adapter, Message Bridge, Message Bus
 - **Message Construction**
 - Command Message, Document Message, Event Message, Request-Reply, Correlation Identifier, Message Sequence,...
 - **Message Routing**
 - Content-based Filter, Message Filter, Process Manager, Message Broker,...
 - **Message Transformation**
 - Envelope Wrapper, Content Enricher, Content Filter,...

PRO-ACTIVITY PROBLEM

gli attori, per loro natura, sono esseri puramente reattivi. Stanno fermi e "dormono" finché non arriva un messaggio nella loro mailbox. Il problema sorge quando vuoi realizzare un comportamento pro-attivo—ovvero un attore che ~~debe~~ portare avanti un piano o un calcolo autonomamente (di sua iniziativa), ma rimanendo comunque pronto a rispondere a messaggi di controllo dall'esterno (com")

- Actors are purely *reactive* entity
 - a main question is: *how to model pro-active behaviours?*
 - that is: behaviours that are oriented to achieve some tasks, by means of some plan of actions
 - *how to properly integrate pro-active and re-active behaviour?*
 - simple example:
 - computing and printing the first 100 prime numbers, eventually stopping if/when receiving a stop message
- (Partial) solutions
 - splitting the actor into multiple interacting actors
 - **message self-sending**

Se un attore si attiva solo quando arrivano messaggi, come facciamo a fargli fare un lavoro lungo e autonomo senza che qualcuno debba "spingerlo" continuamente con dei messaggi?

Invece di avere un unico attore che fa tutto, si crea una gerarchia:

Master Actor: Gestisce la logica di controllo (riceve lo Stop).

Worker Actor: Esegue il calcolo pesante.

L'attore calcola il prossimo numero primo, poi invia un messaggio a se stesso (es. NextStep) e termina l'esecuzione dell'handler corrente. Inviando un messaggio a se stesso, l'attore "prenota" il prossimo turno di lavoro nella sua mailbox. Vantaggio: Tra il calcolo di un numero e l'altro, il framework può inserire altri messaggi (come lo Stop). Se l'attore riceve lo Stop, può cambiare stato (become(Stopped)) e ignorare i futuri messaggi di NextStep che lui stesso si era inviato.

EXPLICIT LOOP / EXPLICIT RECEIVE

L'approccio Explicit Receive elimina gli event-loop nascosti. Il programmatore dice esattamente quando ascoltare la mailbox tramite la primitiva receive e quali messaggi accettare tramite il pattern matching selettivo.

- **No explicit control architecture (and implicit event loops)**
 - **receiving loops must be explicitly handled by programmers**
 - **receive primitive**
 - typically provided with some support for a selective receive with guards
 - this approach is the like direct asynchronous message passing discussed previously
- Concrete examples
 - **Erlang language**

Erlang: (linguaggio funzionale) concorrente, scambio di messaggi esplicito, si ispira agli attori (chiamati processi nel linguaggio) che hanno la receive

Nei framework ad attori orientati agli oggetti (come Akka o ActorFoundry), l'event-loop dell'attore è implicito: la piattaforma gira in background e invoca automaticamente i tuoi metodi/handler (es. onReceive) ogni volta che arriva un messaggio. Nel modello ad Explicit Loop / Explicit Receive (il cui esempio celebre è il linguaggio Erlang), questa "magia" non c'è: è il programmatore a dover gestire esplicitamente il ciclo di vita dell'attore e l'estrazione dei messaggi dalla mailbox.

TASTE OF ACTORS IN ERLANG

Attore che funge da conto corrente

4 tipi di
messaggi che
si possono
ricevere

```
account(Balance) ->
  receive primitiva con cui ci si mette in attesa di messaggi (bloccante)
    {deposit, Amount, Whom} ->
      Whom ! {deposit_receipt, Amount},
      account(Balance + Amount); chiamata ricorsiva tail in cui cambio il balance
                                   (negli attori, sarebbe una sorta di become: entro
                                   in un nuovo stato in cui balance è cambiato)
    {balance, Whom} ->
      Whom ! {balance, Balance},
      account(Balance);
    {withdrawal, Amount, Whom} when Amount > Balance ->
      Whom ! overdraft,
      account(Balance); Guardia (messaggio è disponibile ad essere
                        processato se rispetta la condizione)
    {withdrawal, Amount, Whom} ->
      Whom ! {withdrawal_receipt, Amount},
      account(Balance - Amount)
  end.

...

Account = spawn(fun () -> account(0) end). inizializzo il processo
                                           con balance 0

Account ! {withdrawal, 200}. non ho il problema dell'asynchronous spaghetti
                             perchè receive è bloccante (in attesa del messaggio)
receive
  {withdrawal_receipt, Amount} -> ok,
  overdraft -> not_ok
end.

Account ! {deposit, 300}.
receive {deposit_receipt, A} -> A end.
```


MORE ABOUT ERLANG

- Functional language providing a native support for concurrent programming
 - based on **processes** and process asynchronous communication through **message passing**
- Developed in Ericsson since 1987 for building telecom applications
 - along with ADA, it can be considered the most used and robust concurrent programming language adopted by the industry
- BEAM concurrent virtual machine
 - BEAM stands for Bogdan/Björn's Erlang Abstract Machine
 - completely abstract / virtual notion of process
 - not related to OS process or OS threads
 - extremely efficient process management
 - hundred of thousands processes can be created on a single host

ERLANG AS A FUNCTIONAL LANGUAGE

- An Erlang program describes a series of *functions*
 - operators as special kind of functions
 - each function uses *pattern matching* to determine which function to execute
 - variables start with upper-case (like Prolog)
 - no global variables

```
fact(0) -> 1;  
fact(N) -> N * fact(N - 1).
```

```
fib(1) -> 1;  
fib(2) -> 1;  
fib(N) -> fib(N-1) + fib(N-2).
```

- Modules are used to package functions
 - example: math.erl source file

```
-module(math).  
-export([fact/1]).  
-export([fib/1]).  
  
fact(0) -> 1;  
fact(N) -> N * fact(N - 1);  
  
fib(1) -> 1;  
fib(2) -> 1;  
fib(N) -> fib(N-1) + fib(N-2);
```

ERLANG AS A FUNCTIONAL LANGUAGE

- Calling functions

```
X = math:fact(100).
```

- Compiling and executing programs (..is calling functions..)

```
Erlang (BEAM) emulator version 5.6.4 [source] [smp:2] [async-threads:0] [kernel-poll:false]
```

```
Eshell V5.6.4 (abort with ^G)
```

```
1> c(math).
```

```
{ok,math}
```

```
2> Res = math:fact(15).
```

```
1307674368000
```

DATA STRUCTURES

- *Tuple* and *list* as primitive structured data structures
 - besides atomic data structures (atoms), such as symbols, constants, numbers and strings
- Tuples
 - record-like ordered structure with a fixed number of elements
 - e.g. `Point = { point, 10, 20 }`
 - support for pattern matching
 - e.g. `{point, X, Y} = Point`
 - X is bound to 10 and Y to 20
- Lists
 - to store a variable number of data items
 - e.g. `ThingsToBuy = [ { apples, 10 }, { pears, 6 }, { juice, 2 } ]`
 - head and tail notation: `[ H | T ]`
 - e.g. `ThingsToBuy = [ {apples, X} | T ]`
 - X is bound to 10 and T to `[{ pears, 6 }, { juice, 2 } ]`

DEFINING PROCESSES (= ACTORS)

- A process (actor in Erlang) is a computational activity whose computational behavior is given by some specific function
- The **spawn** primitive to launch a process, getting its PID
 - specifying the function module, function name and parameters

```
Pid = spawn(math, fact, [999]).
```

- Processes are *logical entities*
 - the BEAM maps logical processes to physical threads
 - > programs can have thousands of processes
 - > process creation is very cheap

EXAMPLE:

```
-module(processes).
-export([max/1]).

%% max(N)
%% Create N processes then destroy them
%% See how much time this takes

max(N) ->
    Max = erlang:system_info(process_limit),
    io:format("Maximum allowed processes:~p~n",[Max]),
    statistics(runtime),
    statistics(wall_clock),
    L = for(1, N, fun() -> spawn(fun() -> wait() end) end),
    {_, Time1} = statistics(runtime),
    {_, Time2} = statistics(wall_clock),
    lists:foreach(fun(Pid) -> Pid ! die end, L),
    U1 = Time1 * 1000 / N,
    U2 = Time2 * 1000 / N,
    io:format("Process spawn time=~p (~p) microseconds~n",[U1, U2]).

wait() ->
    receive
        die -> void
    end.

for(N, N, F) -> [F()];
for(I, N, F) -> [F()|for(I+1, N, F)].
```

```
1> processes:max(20000).
Maximum allowed processes:32768
Process spawn time=5.5 (9.4) microseconds
ok
```

ASYNCHRONOUS MESSAGE PASSING

- Processes (actors) can communicate solely through message passing
 - each process has a mailbox, where msg are enqueued
- Sending messages: ! operator

- syntax:

```
Pid ! Message
```

- Receiving messages: **receive** construct

- specifying a pattern
- syntax:

```
receive  
  Pattern1 [when Guard1 ] -> Expression1;  
  Pattern2 [when Guard2 ] -> Expression2;  
  ...  
end
```

- semantics: blocked until a msg in the message queue is found matching a pattern and the corresponding guard is true

A SIMPLE EXAMPLE

```
-module(area_server0).  
-export([loop/0]).  
  
loop() ->  
    receive  
        { rectangle, Width, Ht} ->  
            io:format("Area of rectangle is ~p~n",[Width*Ht]),  
            loop();  
        { circle, R } ->  
            io:format("Area of rectangle is ~p~n",[3.14159*R*R]),  
            loop();  
        Other ->  
            io:format("I don't know what the area of a ~p is ~n",[Other]),  
            loop()  
    end.
```

```
20> c(area_server0).  
{ok,area_server0}  
21> Pid = spawn(fun area_server0:loop/0).  
<0.79.0>  
22> Pid ! {rectangle, 3, 4}.  
Area of rectangle is 12  
{rectangle,3,4}  
23> Pid ! {circle,1}.  
Area of rectangle is 3.14159  
{circle,1}  
24> Pid ! {triangle,1,4}.  
I don't know what the area of a triangle is  
{triangle,1,4
```


“COUNTER” EXAMPLE IN ERLANG

```
-module(counter).  
-export([start/0]).  
  
start() -> loop(0).  
loop(Sum) ->  
    receive  
        {inc} ->  
            loop(Sum+1);  
        {getValue, Pid} ->  
            Pid ! {count_value, Sum},  
            loop(Sum)  
    end.
```

```
-module(counter_user).  
-export([start/2]).  
  
start(Counter,N) -> loop(Counter,0,N).  
  
loop(_,N,N).  
loop(Counter,I,N) ->  
    Counter ! {inc},  
    loop(Counter,I+1,N).
```

```
1> Pid = spawn(counter,start,[ ]),  
    spawn(counter_user,start,[Pid,100]), spawn(counter_user,start,[Pid,100]).
```

- Note the “everything is a process” philosophy
 - no shared memory, then a counter is a process...

ACTORS IN THE MAINSTREAM

- Some links
 - Akka technology
 - Enterprise level
 - Book: “Reactive Messaging Patterns with the Actor Model” (V. Vernon) [VER16]

BIBLIOGRAPHY

- **[HEW-77]** C. Hewitt. Viewing Control Structures as Pattern of Passing Messages. Journal of Artificial Intelligence, 8(3):323-364, 1977
- **[AGH-86]** Gul Agha. Actors: A model of concurrent computation in distributed systems. MIT Press, 1986.
- **[AGHA-90]** Gul Agha, “Concurrent Object Oriented Programming”, Communication of ACM, Sept 1990, Vol 33, No 9
- **[YON-87]** Yonezawa, Tokoro Editors. “Concurrent Object Oriented Programming”. MIT Press. 1987
- **[KAR-09]** Rajesh K. Karmani, Amin Shali, Gul Agha, “Actor Frameworks for the JVM Platform: A Comparative Analysis” - PPPJ’09 - 2009
- **[ACTORS]** Rajesh K. Karmani, Gul Agha: Actors. Encyclopedia of Parallel Computing 2011: 1-11
– <http://www.cs.ucla.edu/~palsberg/course/cs239/papers/karmani-gha.pdf>
- **[ERLANG]** Joe Armstrong. 2010. Erlang. Commun. ACM 53, 9 (September 2010), 68-75. DOI=10.1145/1810891.1810910 <http://doi.acm.org/10.1145/1810891.1810910>
- **[RS-12]** A. Ricci and A. Santi. 2012. Programming abstractions for integrating autonomous and reactive behaviors: an agent-oriented approach. In Proceedings of AGERE! '12. ACM, New York, NY, USA, 83-94.
- **[AGERE]** AGERE! Workshop at SPLASH Conference - 2011, 2012, 2013, 2014, 2015, 2016
 - agere2011.apice.unibo.it, agere2012.apice.unibo.it, <http://agents.usask.ca/agere2013/>
- **[VER16]** V. Vernon. Reactive Messaging Patterns with the Actor Model. Addison Wesley
- **[Schu12]** D. Schumacher. “Actor Idioms” - <http://dl.acm.org/citation.cfm?id=2414655>
- **[Hewitt’s pure perspective]** Actor Model of Computation: Scalable Robust Information Systems